

# SPI Bus Communication Experiment

## SPI Bus Communication Experiment

- 1. SPI Overview
- 2. Core Concepts
  - 2.1 SPI Four-Wire System
  - 2.2 SPI Modes (CPOL & CPHA)
  - 2.3 DMA and Buffer Alignment
  - 2.4 Native IO MUX vs GPIO Matrix
- 3. ESP32-S3 SPI Hardware Specifications
- 4. Project Architecture and Flow
  - Task Topology
  - Data Flow
- 5. Hardware Dependencies
- 6. Source Code Details
  - 6.1 Header Files and Macro Configuration
  - 6.2 spi\_master\_init() — Master Initialization
  - 6.3 spi\_slave\_init() — Slave Initialization
  - 6.4 vSpiSlaveTask() — Slave Full-Duplex Transmit/Receive Task
  - 6.5 app\_main() — Entry Point
- 7. Operation Flow
  - 7.1 Build and Flash Process
  - 7.2 Normal Communication Log
  - 7.3 Four Abnormal Outputs and Causes
    - Case 1: SLV\_MOSI Not Connected (Slave Receive Line Disconnected)
    - Case 2: SLV\_MISO Not Connected (Slave Transmit Line Disconnected)
    - Case 3: SLV\_SCK Not Connected (Clock Line Disconnected)
    - Case 4: SLV\_CS Not Connected (Chip Select Line Disconnected)
    - Summary Comparison of Four Cases
    - Supplement: Master Using Non-Native Pins
- 8. FAQ

## 1. SPI Overview

**SPI (Serial Peripheral Interface)** is a **synchronous full-duplex** serial communication protocol that transfers data at high speed between master and slave through 4 signal lines. Each SPI transfer has master and slave **simultaneously** sending and receiving data (full-duplex).

Typical use cases:

| Scenario        | Description                                  |
|-----------------|----------------------------------------------|
| Flash / SD Card | High-speed data storage                      |
| Display Screens | TFT LCD, OLED (SPI interface)                |
| Sensors         | Accelerometer, gyroscope, temperature sensor |

| Scenario      | Description                                                   |
|---------------|---------------------------------------------------------------|
| Communication | High-speed data exchange between MCUs ( <b>this example</b> ) |

This project demonstrates ESP32-S3 internal **SPI2 (master)** ↔ **SPI3 (slave)** full-duplex communication. A single SPI transfer simultaneously completes bidirectional data exchange without handshaking or synchronization phases.

## 2. Core Concepts

### 2.1 SPI Four-Wire System

| Signal                     | Direction    | Function                                       |
|----------------------------|--------------|------------------------------------------------|
| SCK (Serial Clock)         | Master→Slave | Clock signal, generated by master              |
| MOSI (Master Out Slave In) | Master→Slave | Master sends, slave receives                   |
| MISO (Master In Slave Out) | Slave→Master | Slave sends, master receives                   |
| CS (Chip Select)           | Master→Slave | Chip select (active low), selects target slave |

**Key Feature:** SPI is full-duplex — driven by SCK, data on MOSI and MISO **transmits simultaneously**, meaning while master sends 1 bit to slave, slave also sends 1 bit to master.

### 2.2 SPI Modes (CPOL & CPHA)

SPI has 4 modes, determined by **CPOL (clock polarity)** and **CPHA (clock phase)**:

| Mode | CPOL | CPHA | SCK Idle Level | Sampling Edge |
|------|------|------|----------------|---------------|
| 0    | 0    | 0    | Low            | Rising edge   |
| 1    | 0    | 1    | Low            | Falling edge  |
| 2    | 1    | 0    | High           | Falling edge  |
| 3    | 1    | 1    | High           | Rising edge   |

**This example uses Mode 0** (CPOL=0, CPHA=0), SCK idle low, rising edge sampling. Master and slave must be set to the same mode.

### 2.3 DMA and Buffer Alignment

SPI uses **DMA (Direct Memory Access)** to automatically move data without CPU byte-by-byte participation. ESP32-S3's DMA requirements:

- Buffer **must** be placed in **internal SRAM** ( `.bss` / `DRAM_ATTR` ), cannot be in PSRAM
- Buffer **must** be 32-byte aligned ( `__attribute__((aligned(32)))` )

If the above requirements are violated, DMA will not report an error, but will **silently read/write incorrect data** (all 0xFF or dirty data).

## 2.4 Native IO MUX vs GPIO Matrix

ESP32-S3's SPI2 (FSPI) has **native IO MUX pins**, signals connect directly to SPI controller without going through GPIO exchange matrix:

| Signal        | Native Pin |
|---------------|------------|
| FSPICLK (SCK) | GPIO 10    |
| FSPIQ (MOSI)  | GPIO 12    |
| FSPID (MISO)  | GPIO 13    |
| FSPICS0 (CS)  | GPIO 11    |

Using native pins achieves the lowest timing deviation. If routing SPI signals to other pins through GPIO Matrix, MOSI timing deviation may cause slave to receive all 0xFF.

## 3. ESP32-S3 SPI Hardware Specifications

| Feature                   | Parameter                                           |
|---------------------------|-----------------------------------------------------|
| SPI Controller Count      | 4 (SPI0/SPI1 for Flash/PSRAM, SPI2/SPI3 for user)   |
| Max Clock Frequency       | 80 MHz (SPI2/SPI3)                                  |
| Mode Support              | 0 / 1 / 2 / 3 (CPOL + CPHA)                         |
| Full-duplex / Half-duplex | Both supported                                      |
| Data Bit Width            | 1 / 2 / 4 / 8 wire                                  |
| DMA Support               | Supported (GDMA channel)                            |
| Master / Slave            | SPI2/SPI3 can both be configured as master or slave |

## 4. Project Architecture and Flow

```
app_main()
├─ memset() Clear 4 DMA buffers
├─ strcpy() Fill mst_tx="Hello from MASTER", slv_tx="SLAVE IS READY!"
├─ spi_slave_init()           SPI3 slave initialization, DMA auto
├─ xTaskCreate(vSpiSlaveTask) Create slave task, priority 10, 4KB stack
├─ vTaskDelay(800ms)         wait for slave ready
├─ spi_master_init()         SPI2 master initialization, 40kHz, Mode 0
├─ while(1)                  Main loop (2 second cycle)
│   ├─ spi_device_transmit() Full-duplex transfer: mst_tx → slave, slave → mst_rx
│   └─ printf() bidirectional results
```

|                        |                                                     |
|------------------------|-----------------------------------------------------|
| vSpiSlaveTask()        | Independent task, runs in parallel                  |
| └ while(1)             |                                                     |
| └ spi_slave_transmit() | Block wait, print slv_rx and slv_tx after receiving |

Task Topology

| Task                          | Priority       | Stack     | Responsibility                                          |
|-------------------------------|----------------|-----------|---------------------------------------------------------|
| slave_task<br>(vSpiSlaveTask) | 10             | 4096<br>B | Slave blocks and waits for full-duplex transmit/receive |
| main (app_main)               | 1<br>(default) | 3584<br>B | Master periodic full-duplex transmit/receive            |

Data Flow

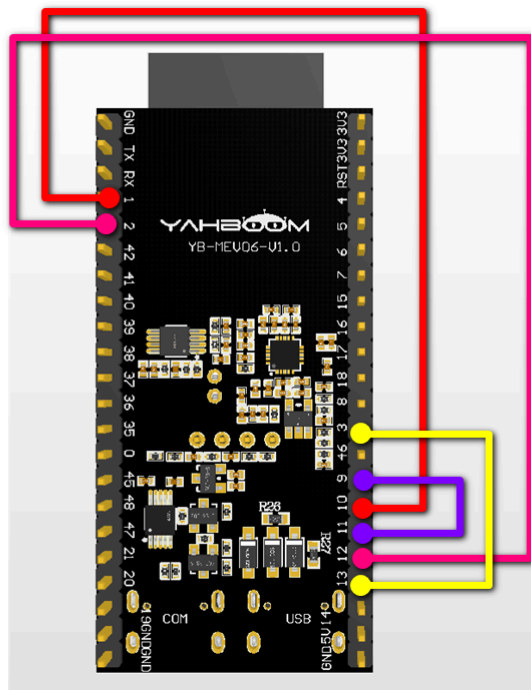
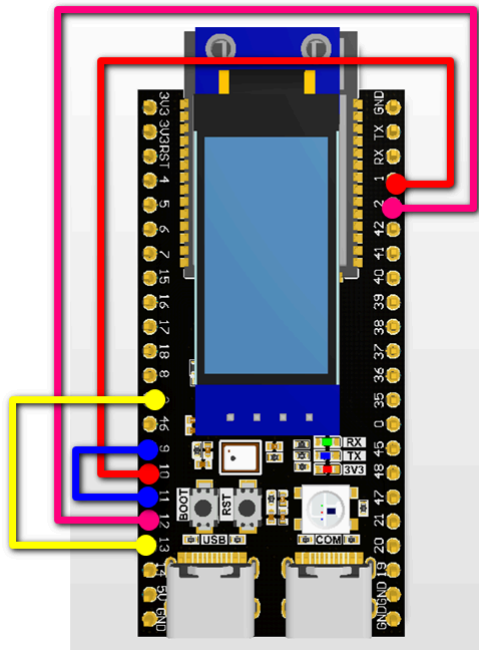
|                                              |                       |               |
|----------------------------------------------|-----------------------|---------------|
| Master mst_tx                                | →MOSI (GPIO12→GPIO2)→ | slave slv_rx  |
| slave slv_tx                                 | →MISO (GPIO3←GPIO13)→ | Master mst_rx |
| (Same SPI transfer completes simultaneously) |                       |               |

5. Hardware Dependencies

Hardware Connection (ESP32-S3 internal pin shorted):

| Signal | Master (SPI2)     | Slave (SPI3) |
|--------|-------------------|--------------|
| SCK    | GPIO 10 (FSPICLK) | GPIO 1       |
| MOSI   | GPIO 12 (FSPIQ)   | GPIO 2       |
| MISO   | GPIO 13 (FSPID)   | GPIO 3       |
| CS     | GPIO 11 (FSPICS0) | GPIO 9       |

**Note:** Master must use FSPI native IO MUX pins. Slave can use any GPIO (avoid PSRAM pins 33~37).



**Required Components:** `driver/spi_master`, `driver/spi_slave`, `FreeRTOS`

## 6. Source Code Details

### 6.1 Header Files and Macro Configuration

This code section includes header file references, pin macro definitions, and declarations of 4 DMA buffers. All buffers meet 32-byte alignment and are placed in internal SRAM.

```
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/spi_master.h"
```

```

#include "driver/spi_slave.h"
#include "esp_log.h"

#define TAG "SPI_DOUBLE"

/* SPI2 Master (FSPI native IO MUX) */
#define MST_HOST      SPI2_HOST
#define MST_SCK        10      // FSPICLK
#define MST_MOSI       12      // FSPIQ
#define MST_MISO       13      // FSPID
#define MST_CS         11      // FSPICS0

/* SPI3 slave */
#define SLV_HOST      SPI3_HOST
#define SLV_SCK        1
#define SLV_MOSI       2
#define SLV_MISO       3
#define SLV_CS         9

#define BUF_SIZE      64

/* DMA buffers: 32-byte aligned + internal SRAM */
static DRAM_ATTR uint8_t mst_tx[BUF_SIZE] __attribute__((aligned(32)));
static DRAM_ATTR uint8_t mst_rx[BUF_SIZE] __attribute__((aligned(32)));
static DRAM_ATTR uint8_t slv_tx[BUF_SIZE] __attribute__((aligned(32)));
static DRAM_ATTR uint8_t slv_rx[BUF_SIZE] __attribute__((aligned(32)));

```

`DRAM_ATTR` ensures variables are placed in internal DRAM (not PSRAM), `aligned(32)` meets DMA alignment requirements. Both are essential.

## 6.2 spi\_master\_init() — Master Initialization

This function initializes SPI2 as SPI master, configures bus pins, clock frequency (40kHz), and SPI mode (Mode 0), finally mounts the device to the bus.

```

static void spi_master_init(void)
{
    spi_bus_config_t bus_cfg = {
        .mosi_io_num = MST_MOSI, .miso_io_num = MST_MISO,
        .sclk_io_num = MST_SCK,
        .quadwp_io_num = -1, .quadhd_io_num = -1,
        .max_transfer_sz = BUF_SIZE,
    };
    spi_bus_initialize(MST_HOST, &bus_cfg, SPI_DMA_CH_AUTO);

    spi_device_interface_config_t dev_cfg = {
        // 40kHz low speed for stability
        .clock_speed_hz = 40000,
        .mode = 0,
        .spics_io_num = MST_CS,
        .queue_size = 1,
    };
}

```

```

spi_bus_add_device(MST_HOST, &dev_cfg, &spi_master);
}

```

## 6.3 spi\_slave\_init() — Slave Initialization

This function initializes SPI3 as SPI slave, configures bus pins, SPI mode (Mode 0), and chip select pin, consistent with master mode.

```

static void spi_slave_init(void)
{
    spi_bus_config_t bus_cfg = {
        .mosi_io_num = SLV_MOSI, .miso_io_num = SLV_MISO,
        .sclk_io_num = SLV_SCK,
        .quadwp_io_num = -1, .quadhd_io_num = -1,
    };

    spi_slave_interface_config_t slv_cfg = {
        .mode = 0,
        .spics_io_num = SLV_CS,
        .queue_size = 1,
        .flags = 0,
    };
    spi_slave_initialize(SLV_HOST, &bus_cfg, &slv_cfg, SPI_DMA_CH_AUTO);
}

```

## 6.4 vSpiSlaveTask() — Slave Full-Duplex Transmit/Receive Task

The slave in an independent FreeRTOS task loops calling `spi_slave_transmit()` to block and wait for master to initiate transfer. Each transfer completes transmit and receive simultaneously:

- `slv_rx` receives data sent by master
- `slv_tx` content is sent to master

The slave task blocks in an independent thread waiting for master to initiate SPI transfer, completes transmit/receive simultaneously each transfer and prints results.

```

void vspiSlaveTask(void *arg)
{
    ESP_LOGI(TAG, "Slave started, waiting for master...");

    while (1) {
        memset(slv_rx, 0, BUF_SIZE);

        spi_slave_transaction_t t = {
            .length = BUF_SIZE * 8,
            .tx_buffer = slv_tx,
            .rx_buffer = slv_rx,
        };
        spi_slave_transmit(SLV_HOST, &t, portMAX_DELAY);

        ESP_LOGI(TAG, "Slave received: %s", slv_rx);
    }
}

```

```

        ESP_LOGI(TAG, "Slave sent:    %s", slv_tx);
    }
}

```

## 6.5 app\_main() — Entry Point

The entry point function sequentially: clear DMA buffers, fill test data, start slave task, initialize master, finally enter main loop to periodically execute full-duplex SPI transfer and print bidirectional results.

```

void app_main(void)
{
    memset(mst_tx, 0, BUF_SIZE);
    memset(mst_rx, 0, BUF_SIZE);
    memset(slv_tx, 0, BUF_SIZE);
    memset(slv_rx, 0, BUF_SIZE);

    strcpy((char *)mst_tx, "Hello from MASTER");
    strcpy((char *)slv_tx, "SLAVE IS READY!");

    // Start slave first, then master
    spi_slave_init();
    xTaskCreate(vSpiSlaveTask, "slave_task", 4096, NULL, 10, NULL);
    vTaskDelay(pdMS_TO_TICKS(800));

    spi_master_init();
    ESP_LOGI(TAG, "Master initialized!");

    while (1) {
        memset(mst_rx, 0, BUF_SIZE);

        spi_transaction_t t = {
            .length      = BUF_SIZE * 8,
            .tx_buffer    = mst_tx,
            .rx_buffer    = mst_rx,
        };

        ESP_LOGI(TAG, "Master sent:    %s", mst_tx);
        spi_device_transmit(spi_master, &t);
        ESP_LOGI(TAG, "Master received: %s", mst_rx);

        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

```

**Why initialize slave first?** Slave must be **ready before master**. If master starts first and sends signal to CS, but slave's `spi_slave_transmit()` is not yet queued, master will receive all 0xFF.



## 7. Operation Flow

### 7.1 Build and Flash Process

For detailed flash process: 1. Before Development → 3. Build and Flash

### 7.2 Normal Communication Log

After wiring according to pin table and powering on, bidirectional communication once every 2 seconds:

```
I (1082) SPI_DOUBLE: Master initialized!
=====
I (1092) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1102) SPI_DOUBLE: Slave received: Hello from MASTER
I (1102) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1102) SPI_DOUBLE: Master received: SLAVE IS READY!
=====
```

```
I (1082) SPI_DOUBLE: Master initialized!
=====
I (1092) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1102) SPI_DOUBLE: Slave received: Hello from MASTER
I (1102) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1102) SPI_DOUBLE: Master received: SLAVE IS READY!
=====
=====
I (3112) SPI_DOUBLE: Master sent:      Hello from MASTER
I (3112) SPI_DOUBLE: Slave received: Hello from MASTER
I (3112) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (3112) SPI_DOUBLE: Master received: SLAVE IS READY!
=====
```

Key verification points:

- Master `my_tx` "Hello from MASTER" → slave `slv_rx` matches
- Slave `slv_tx` "SLAVE IS READY!" → master `mst_rx` matches

### 7.3 Four Abnormal Outputs and Causes

Each of the four SPI signal lines has its own function. Any one disconnected causes different abnormal phenomena. Below analyzes each case of one line disconnected, the serial output and root cause.

#### Case 1: SLV\_MOSI Not Connected (Slave Receive Line Disconnected)

**Phenomenon:** Master sends normally, slave receives all `0xFF`; slave sends normally, master correctly receives slave data.

**Cause:** MOSI is the only channel for slave to receive data. After disconnection, slave MOSI pin floats (internal pull-up to high), SCK clock samples logic `1` each time, so slave receives `0xFF` for each byte. But MISO is still normally connected, so master can correctly receive data sent by slave.

**Serial output example:**

```
I (1082) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1092) SPI_DOUBLE: Slave received:      ← Received 0x00
I (1092) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1092) SPI_DOUBLE: Master received: SLAVE IS READY!      ← Master received normally
```

```
=====
I (1082) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1092) SPI_DOUBLE: Slave received:
I (1092) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1092) SPI_DOUBLE: Master received: SLAVE IS READY!
=====
```

## Case 2: SLV\_MISO Not Connected (Slave Transmit Line Disconnected)

**Phenomenon:** Master sends normally, slave correctly receives master data; slave sends normally, but master receives all `0xFF`.

**Cause:** MISO is the only channel for master to receive slave data. After disconnection, master MISO pin floats (internal pull-up to high), SCK clock samples logic `1` each time, so master receives `0xFF` for each byte. But MOSI is still normally connected, so slave can correctly receive data sent by master.

**Serial output example:**

```
I (1092) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1092) SPI_DOUBLE: Slave received: Hello from MASTER      ← Slave received normally
I (1092) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1102) SPI_DOUBLE: Master received:      ← Received 0x00
```

```
=====
I (1082) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1092) SPI_DOUBLE: Slave received: Hello from MASTER
I (1092) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1102) SPI_DOUBLE: Master received:
=====
```

## Case 3: SLV\_SCK Not Connected (Clock Line Disconnected)

**Phenomenon:** Both tasks run normally, but transmit/receive data is **empty** (buffer keeps original value after `memset` clear, all `0x00`).

**Cause:** SCK is the clock signal that drives data shifting. After wire break, although CS correctly "selects" both sides into transfer state, without SCK transitions, data **is not shifted out or sampled** — shift register stays still. When CS deselects, `spi_slave_transmit()` still returns normally (triggered by CS **rising edge**), but both sides' `rx_buffer` contains `0x00` after `memset`, so printed as empty string.

**Key difference:** Slave does **not block** after SCK disconnected. ESP-IDF's `spi_slave_transmit()` is triggered to complete by CS signal's rising edge, not waiting for SCK transition. But data shifting must rely on SCK, so after CS pulls low→high, transfer "formally completes", actually no data flows.

**Serial output example:**

```
I (1082) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1092) SPI_DOUBLE: Slave received:
I (1092) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1092) SPI_DOUBLE: Master received:
```

```
I (1082) SPI_DOUBLE: Master initialized!
=====
I (1082) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1092) SPI_DOUBLE: Slave received:
I (1092) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1092) SPI_DOUBLE: Master received:
=====
```

## Case 4: SLV\_CS Not Connected (Chip Select Line Disconnected)

**Phenomenon:** Communication **may be completely normal**, slave correctly transmits/receives; or slave may not respond at all. Depends on floating level.

**Cause:** CS (Chip Select) is the signal line master uses to select target slave, **active low** (slave is selected when CS = 0). After CS pin is disconnected, it is in **floating state**, its input level is uncertain:

| Floating Level                | Slave State     | Communication Result                            |
|-------------------------------|-----------------|-------------------------------------------------|
| Float low ( $\approx 0V$ )    | Always selected | Communication normal (no different from wiring) |
| Float high ( $\approx 3.3V$ ) | Never selected  | Slave does not respond (blocks waiting)         |

Floating level of GPIO pin when floating is affected by:

- **Internal weak pull-down/pull-up resistor:** ESP32-S3 IO pins have configurable weak pull-up/pull-down, default state after reset may make pin tend toward a certain level
- **Parasitic capacitance of PCB traces:** Adjacent high-speed signal lines (such as SCK) may affect floating pin through capacitive coupling
- **Environmental noise:** EMI can also affect floating pin's instantaneous level

**Measured conclusion:** Whether communication is normal after CS disconnected is **uncertain**, depends on specific hardware environment and floating level. In this single-slave scenario, as long as CS happens to float low, communication will work normally. This is also why CS wire can sometimes be omitted in single-slave systems — but when slave floats high it still blocks, **do not rely on floating behavior in production environments**.

**Serial output example when floating low** (communication normal):

```
I (1092) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1102) SPI_DOUBLE: Slave received: Hello from MASTER      ← Normal! Slave "always selected"
I (1102) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1102) SPI_DOUBLE: Master received: SLAVE IS READY!      ← Normal!
```

```
=====
I (1082) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1082) SPI_DOUBLE: Slave received: Hello from MASTER
I (1082) SPI_DOUBLE: Slave sent:      SLAVE IS READY!
I (1092) SPI_DOUBLE: Master received: SLAVE IS READY!
=====
```

Serial output example when floating high (communication failed):

```
I (1092) SPI_DOUBLE: Master sent:      Hello from MASTER
I (1102) SPI_DOUBLE: Master received:          ← Master received 0xFF
                                              ← Slave has no output!
```

Summary Comparison of Four Cases

| Disconnected Signal    | Master TX → Slave RX                                  | Slave TX → Master RX                          | Slave Task Running        |
|------------------------|-------------------------------------------------------|-----------------------------------------------|---------------------------|
| SLV_MOSI not connected | Slave receives 0x00                                   | Normal                                        | Runs normally             |
| SLV_MISO not connected | Normal                                                | Master receives 0x00                          | Runs normally             |
| SLV_SCK not connected  | Slave receives empty (0x00)                           | Master receives empty (0x00)                  | Runs normally             |
| SLV_CS not connected   | Uncertain (float low=normal, float high=slave blocks) | Uncertain (float low=normal, float high=0x00) | Depends on floating level |

**Tip:** These four abnormal outputs are effective tools for locating SPI hardware wiring problems — based on "which side receives 0xFF" you can quickly locate the disconnected signal line.

Supplement: Master Using Non-Native Pins

Routing through GPIO Matrix introduces additional delay, slave may receive incorrect data. Always use FSPI native pins (10/12/13/11).

8. FAQ

| Symptom                      | Cause                                               | Solution                                                                              |
|------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------|
| Master receives all 0xFF     | Slave not started / MISO floating                   | Ensure <code>spi_slave_init()</code> and <code>vspiSlaveTask</code> run before master |
| Slave receives all 0xFF      | MOSI timing deviation / master uses non-native pins | Master use FSPI native IO MUX pins (GPIO 10/12/13/11)                                 |
| Both sides have 0xFF         | 4 SPI wires not connected                           | Check SCK, MOSI, MISO, CS four Dupont wires                                           |
| Data occasionally misaligned | Clock frequency too high / cable too long           | Lower <code>clock_speed_hz</code> (e.g., 10000), shorten Dupont wires                 |

| Symptom                                            | Cause                          | Solution                                            |
|----------------------------------------------------|--------------------------------|-----------------------------------------------------|
| Build error<br><code>spi_master.h</code> not found | SPI driver not enabled         | menuconfig → Component config → Driver → SPI driver |
| DMA buffer data abnormal                           | Buffer not aligned or in PSRAM | Check <code>DRAM_ATTR + aligned(32)</code>          |